

Deception Detection

Saurav Mehra
IIIT-Delhi

Ankur Bohra
IIIT-Delhi

Kartikeya Sehgal
IIIT-Delhi

{saurav22465,ankur22079,kartikeya22244}@iiitd.ac.in

1 Introduction

Deception refers to the act of intentionally leading someone to believe false information. In strategy-based games like Mafia and Diplomacy, where players employ deception and misdirection to mislead the other players to gain an advantage, the ability to detect the true intentions of the opposite player becomes essential. This challenge has been framed as a natural language processing (NLP) task known as deception detection, where various computational techniques are employed to distinguish between the surface meaning and the underlying intent of a message or conversation.

Diplomacy is a strategic war-game set in Europe in the years leading up to World War I. Players act as the major European powers and are able to expand their territories on the continent. The game features a limited number of *supply centers* that produce units necessary for military expansion. Thus, players must form and break alliances to expand their territories until they control over half the total number of supply centers. Deception is built into the game by design, and diplomacy players pride themselves on their ability to both detect and tell believable lies.

Players engaging in deception leave subtle linguistic clues in their phrasing while hiding their true motives. In such cases, the linguistic cues, sentiments, and structure of the conversation can serve as important features for deep learning-based detection, inherently making it a natural language processing task.

This project, therefore, aims to develop a model capable of predicting deception in messages exchanged between players in the game of **Diplomacy**, by not only exploring the in-game conversations between the players but also the moves played by them on the map, in a manner similar to how a human observes the game.

2 Related Work

Initially, deception detection was done at an utterance level (Newman et al., 2003), without using any context of where the ‘message’ or ‘text’ is being spoken. A deceptive sentence can seem just like a normal sentence, but the deception portion becomes highlighted when considered a part of a discourse or impacts the following sentence.

Niculae et al. (2015) study the social phenomenon of betrayal in dialogues by collecting and themselves labelling conversations between players in the online game Diplomacy. The setting of Diplomacy provides a new angle to the task of deception detection, where no player has a fixed role, and any player at any point during the game may employ deception as a strategy. The task of detecting deception now expands to analyzing conversational texts, rather than merely identifying a fixed anchor or individual who is lying. Niculae et al. (2015) conclude that subtle conversational patterns arise before one player betrays the other. Specifically, the authors find changes in sentiment, increases in argumentation and discourse, politeness, subjectivity, and talkativeness to be ‘linguistic harbingers’ of betrayal.

Peskov et al. (2020) note that Niculae et al. (2015)’s dataset was ‘limited to *post-hoc* analysis’. The intention to deceive being a highly subjective phenomenon, the authors hypothesize that having players label their own messages was more reflective of their internal state. Therefore, they develop an interface for players to provide their intentions and perceptions. Notably, they also provide a complete recording of the game in the form of the moves played in each game. Deviating from Niculae et al. (2015) analytical approach to deception, Peskov et al. (2020) tackle the problem of deception detection itself. Towards this goal, they develop several baselines incorporating different aspects of the data: Niculae et al.

(2015)’s Harbingers, conversational context, and also Diplomacy-specific features such as power imbalance between the speakers (the difference in the number of supply centers of the communicating powers).

We explore the use of graphs in natural language processing to understand how data can be modeled as a graph and then passed into a neural network along with other contextual embeddings. Graph Attention Networks (GATs), introduced in (Veličković et al., 2018), provide a graph-based learning method that utilizes masked self-attention to dynamically assign importance weights to neighboring nodes. This enables a better representation of the data—one that is more efficient and robust. GATs outperformed the then-popular Graph Convolutional Networks (GCNs) on several benchmark tasks, demonstrating the advantages of attention mechanisms in graph-based learning.

3 Methodology

3.1 Baselines

The **Bag-of-Words** (BoW) approach treats text as an unordered collection of words, disregarding sequence but maintaining word frequency. Logistic regression is performed on the numerical features, thus extracted from the text.

The **LSTM** model utilizes the GloVe embeddings trained on 27 billion tokens from Twitter, with a vocabulary size of 1.2 million and 200-dimensional vectors.

The baselines have been implemented by appropriately replicating the code¹ provided by Peskov et al., 2020.

Baseline	Macro F1	Weighted F1	Accuracy
BoW	0.534	0.828	0.806
LSTM	0.521	0.830	0.814

Table 1: Comparison of BoW and LSTM models.

3.2 Our Approach

Our literature review reveals that existing papers within our problem scope do not consider the moves or the board state provided in the data. Incorporating player moves would indicate the players’ intent and help identify the deceptive messages.

We propose using graphs to model adjacent territories (nodes) and their current game state (active power, units placed, and other metadata). For this

purpose, we pre-process the moves provided to create their graphical representation.

3.2.1 Preprocessing Moves

We first create a template map of all the territories and their adjacent provinces, with additional information at each node (territory) such as whether it has a supply center or not, whether there is an army unit or a fleet unit, and the name of the country under whose control it is at the moment. Using this template as the base, we process all the move files and save their corresponding maps for further use.

3.2.2 Preprocessing Messages

The in-game conversations between the players, provided as messages, were cleaned by removing additional whitespaces, links, and emojis². The messages were then normalized by converting to lowercase.

3.2.3 Processing the Dataset

The Dataset class loads in data as conversations between two players for a specific game, along with the associated metadata. The vocabulary is built using the messages in the training set. For each message, the *game_id*, *year* and *season* are also obtained in order to load the corresponding map state and the preceding moves at that point in time. This is done for the entire conversation, which is treated as a single unit and these conversations are then processed in batches.

3.3 Model Overview

For a particular message between two players, the moves made by both players in the immediately previous adjudication (or *no move* when the game begins) extracted and processed as sentences. Their GloVe (27 Billion, 200 dimensions) embeddings are obtained and then fed through an encoder.

The state of the map when the message was communicated is obtained using the generated maps as discussed in the preceding subsections. Each province or node is represented as a concatenation of one-hot vectors, based on the type of province it is, whether it is a supply center or not, and who controls that province currently, among other details. This feature vector is fed into an encoder that learns a relatively dense representation of these nodes. From these encoded vectors, a spatial graph is constructed, with edges added between provinces

¹https://github.com/DenisPeskov/2020_acl_diplomacy

²<https://stackoverflow.com/questions/33404752/removing-emojis-from-a-string-in-python>

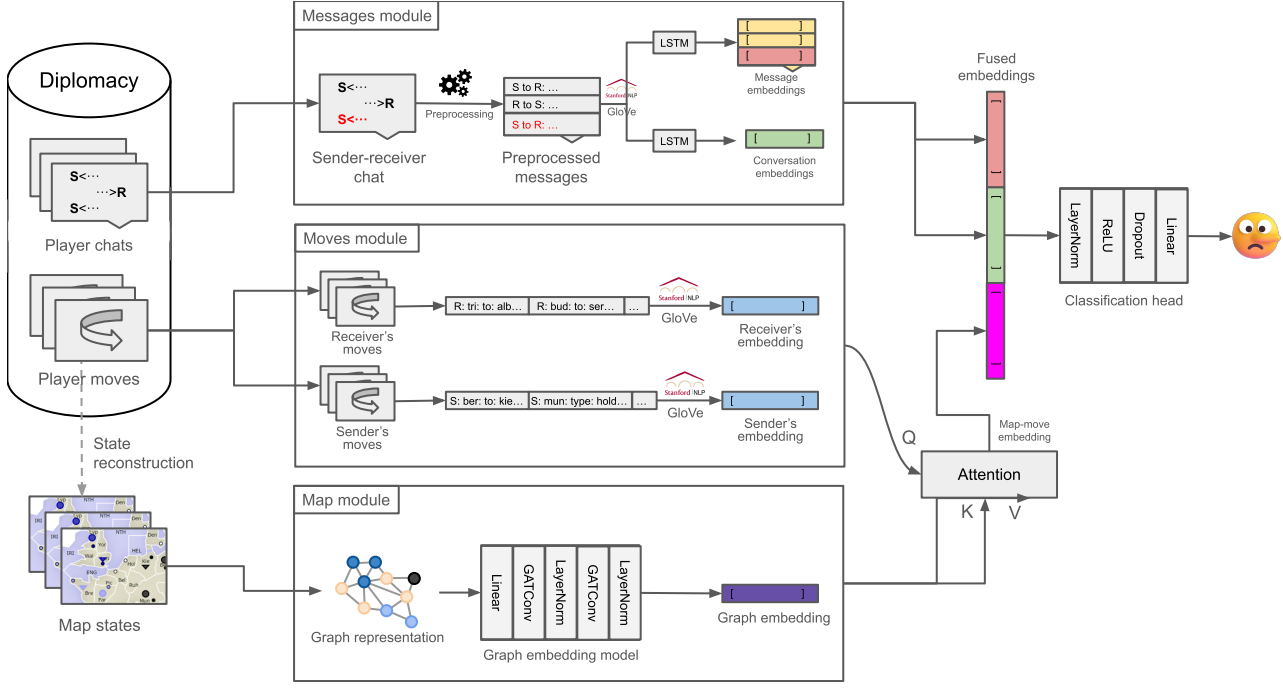


Figure 1: Model Architecture

(or their encoded representations), that are adjacent to each other on the actual Diplomacy map. This graph is passed through a graph neural network with the to obtain the relevant embeddings for each node.

Using cross-modal attention, the embeddings of the moves made by the 2 players (the speaker and the receiver) are used to align the embeddings of the map, with the intent to obtain embeddings of regions that are relevant to them currently based on their previous moves.

These attention outputs are fused together with the current message embeddings obtained through a bidirectional LSTM, along with the previous context obtained through a unidirectional LSTM, encoding all the messages in the conversation of the two players so far. These fused embeddings are then used to obtain logits for making predictions.

4 Dataset Description

The *Diplomacy* dataset contains data from 12 games played on the Backstabber platform. These games span a period of 10 years, from 1901 to 1910. The data for each year is divided into three parts according to the season: fall, spring, and winter. The data for all of the games has been split into training, validation, and test sets in the form of JSONL files. The test set contains data from Games 4 and 12, while the validation set consists

of data from Game 11. The remaining data is part of the training set.

Each JSON object in the file contains the following attributes:

- **speakers:** A list of strings representing the player names who sent the message at the corresponding index (i.e., each item is one of the 7 countries).
- **receivers:** A list of strings representing the player names who received the message at the corresponding index (i.e., each item is one of the 7 countries).
- **messages:** The list of raw message strings (string format; ranges in length from one word to paragraphs in length).
- **sender_labels:** Corresponds to the ground truth label, annotated by the sender of the message itself to identify whether what they said was true or false.
- **receiver_labels:** This is the suspected or assumed meaning of what the receiver perceived the sender's message to be (boolean variable).
- **game_score:** The current game score—supply centers—of the sender (string format, ranges from 0 to 18).

- **score_delta**: The current game score—supply centers—of the sender minus the game score of the recipient (string format, ranges from -18 to 18).
- **absolute_message_index**: The index of the message in the entire game, across all dialogs (integer format).
- **relative_message_index**: The index of the message in the current dialog (integer format).
- **seasons**: The season in Diplomacy, associated with the year (string format: Spring, Fall, Winter).
- **years**: The year in Diplomacy, associated with the season (string format: 1901 through 1918).
- **game_id**: Which of the 12 games the dialog comes from (integer format, ranging from 1 to 12).

The above description of the variables has been taken from https://github.com/DenisPeskoff/2020_acl_diplomacy/tree/master/data

The training set comprises 13,132 messages from nine games. The validation set includes 1,416 messages from a single game, and the test set contains 2,741 messages from two games.

The above data correspond only to the in-game messages exchanged between players. The authors also provided a set of files corresponding to the moves made by the players. A JSON file corresponds to the moves for each season of each year. In total, 342 such files were provided. However, the data set is highly imbalanced, with only 5% statements labeled as lies.

We plot the map of the diplomacy game for reference while processing the moves in Fig. 2.

5 Experimental Setup

The following metrics were considered for evaluation:

- **Macro F1-Score**: The average of individual F1-Scores of each class.
- **Accuracy**: This measures the total number of correct predictions; however, in our case, it is often inflated since it always predicts truth due to class imbalance in the dataset.

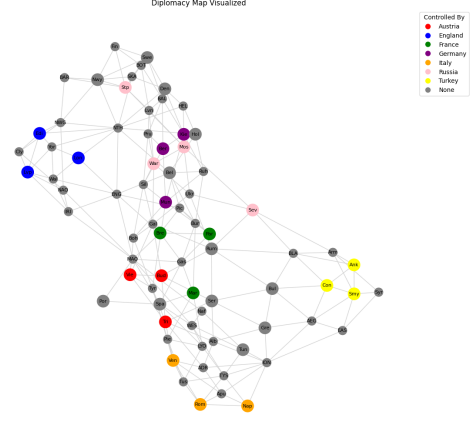


Figure 2: Diplomacy Map

All experiments were conducted on a Kaggle environment equipped with dual T4 GPUs.

- We experimented with different batch sizes—8, 4, and 2—to observe their impact on model performance.
- The AdamW optimizer was employed with a learning rate of $5e-4$ and a weight decay of 0.001.
- Binary classification was performed using the cross-entropy loss function. To address the class imbalance, appropriate class weights were assigned.
- Dropout was also experimented with in the model architecture to mitigate overfitting.
- We initially experimented with 30 epochs, but since the model tended to converge halfway through, all subsequent iterations were trained for only 15 epochs.

6 Results

Our model achieves a macro-F1 score of 0.55 on the test set. It has a weighted-F1 score of 0.85 and an accuracy of 0.85. Its confusion matrix can be seen in Table 2.

	Predicted Lie	Predicted Truth
Actual Lie	43	197
Actual Truth	210	2288

Table 2: Model Confusion Matrix

7 Analysis

The model is able to categorize the true statements quite confidently. However, it struggles with identifying lies, as indicated by the high false negatives and positives.

Although this model is unable to match few of the models’ macro F1-scores outlined by [Peskov et al. \(2020\)](#), as part of our ablation analysis, we tested the performance of the model without the graph and move embeddings. The only input given to the model was the message and the previous context, with the exact same configuration. This implementation only managed to achieve a macro-F1 score of 0.52. From its confusion matrix (Tab. 3), it can be observed that this model is much weaker at identifying lies and mostly categorizes statements as true due to the class imbalance. This clearly suggests that incorporating this additional data, the map and the moves, from the game boosts the performance of the model, aiding in distinguishing lies from the abundance of true statements.

	Predicted Lie	Predicted Truth
Actual Lie	27	213
Actual Truth	199	2299

Table 3: Context Only Model Confusion Matrix

8 Conclusion and Future Work

It can be observed that incorporating other meta-data into the *Deception Detection* model for Diplomacy enhances performance compared to a model that simply looks for cues in textual data. Utilizing temporal graphs, to see how players are moving over time, may lead to even better performance.

Although messages, moves, and the map are all used by the model, cross attention is restricted between the move and map embeddings. It is likely that detection would benefit from being able to align claims in the text with intentions indicated by the moves and the current state of the map.

With our initial aim being to make the model observe all data from the game at once, similar to a human, a key component is still missing. A player engages in conversation with other players in the game apart from the speaker as well. Humans utilize those conversations in conjunction with all the other information in order to classify the statement. Incorporating these conversations into the previous context could potentially further improve

performance. Additionally, a more enhanced and expansive dataset is likely to enhance the models, as was observed by [Peskov et al. \(2020\)](#).

No-Press Diplomacy, a variant in which players do not communicate with each other through text, often utilizes Reinforcement Learning to train agents for playing this game. Combining these agents, which aim to understand player behavior in the context of the game, with textual information could aid in this deception detection.

References

- Matthew L Newman, James W Pennebaker, Drew S Berry, and Jon M Richards. 2003. [Lying words: predicting deception from linguistic styles](#). *Personality and Social Psychology Bulletin*, 29(5):665–675.
- Vlad Niculae, Srijan Kumar, Jordan Boyd-Graber, and Cristian Danescu-Niculescu-Mizil. 2015. [Linguistic harbingers of betrayal: A case study on an online strategy game](#). In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1650–1659, Beijing, China. Association for Computational Linguistics.
- Denis Peskov, Benny Cheng, Ahmed Elgohary, Joe Barrow, Cristian Danescu-Niculescu-Mizil, and Jordan Boyd-Graber. 2020. [It takes two to lie: One to lie, and one to listen](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 3811–3854, Online. Association for Computational Linguistics.
- Daniel P. Stormont and Vicki H. Allan. 2012. A comparison of diplomacy gameboard graph search algorithms. In *ICAART 2012 - Proceedings of the 4th International Conference on Agents and Artificial Intelligence, Volume 2 - Agents, Vilamoura, Algarve, Portugal, 6-8 February, 2012*, pages 371–374. SciTePress.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. [Graph attention networks](#). *Preprint*, arXiv:1710.10903.
- Zhe Zhao Zhiruo Wang Qi Ju Haotang Deng Ping Wang Weijie Liu, Peng Zhou. 2020. K-BERT: Enabling language representation with knowledge graph. In *Proceedings of AAAI 2020*.